

PSE-VALIDATION-RUNNER-01

One-Button Validation Singularity

Formale Implementierungsspezifikation fuer deterministische Gesamtvalidierung v0.1

Sebastian Klemm

Technische Ausarbeitung fuer Coding-Agent-Implementierung

Mai 2026

Zusammenfassung

Diese Spezifikation definiert einen Validierungsrunner fuer das aktuelle PSE-Repository. Ziel ist ein einzelner Bedienpunkt, der Build, Tests, Replays, interne Eval-Matrix-Presets, Benchmarks, optionale reale Datensätze, Ablationen, Reports, Artefaktbündel und Ergebnisurteile deterministisch ausführt. Der Runner ist keine neue Kognitionsschicht. Er ist die reproduzierbare Mess- und Schließungsmaschine des Systems: ein Knopf erzeugt einen vollständigen Validierungsordner, der alle Befunde, Logs, Hashes, Reports, Metriken, offenen Proof-Obligations und ein finales Urteil enthält.

Inhaltsverzeichnis

1	Status und Ziel	3
1.1	Status	3
1.2	Problem	3
1.3	Ziel	3
2	Nicht-Ziele	3
3	Systemposition	4
3.1	Prinzip	4
4	Konformitätsklassen	4
5	Ein-Knopf-Bedienmodell	5
5.1	Primärer Befehl	5
5.2	Profile	5
5.3	Subcommands	5
6	Artefaktstruktur	5
7	Datenmodelle	6
7.1	ValidationProfile	6
7.2	RepositorySnapshot	6
7.3	EnvironmentFingerprint	7
7.4	ValidationCommand	7
7.5	ValidationRunLedger	7
8	Ausführungsphasen	8
8.1	Phase L0: Snapshot und Quality Gate	8
8.2	Phase L1: Core-Benchmarks	8
8.3	Phase L2: Eval-Matrix-Presets	9
8.4	Phase L3: Optionale reale Domänenvalidierung	9
9	Validierungsurteile	9
9.1	Schlussformel	9
10	Metrikgruppen	10
11	Runner-Architektur	10
11.1	Crates und Tools	10
11.2	Feature Flags	11

12 Report-Schichten	11
12.1 FinalReport JSON	11
12.2 FinalReport Markdown	11
13 Fehlerklassen	12
14 Erste reale Domänenprofile	12
14.1 Empfohlene Reihenfolge	12
14.2 Minimaler DatasetManifest	12
15 Tests	12
15.1 Unit Tests	12
15.2 Integration Tests	13
15.3 CLI Smoke Tests	13
16 Definition of Done	13
17 Coding-Agent-Auftrag	13
18 Schlussformel	14

1 Status und Ziel

1.1 Status

Dieses Dokument ist eine normative Implementierungsspezifikation. Die Begriffe **MUST**, **MUST NOT**, **SHOULD** und **MAY** markieren verbindliche Anforderungen.

1.2 Problem

Das PSE-Repository besitzt inzwischen mehrere implementierte Schichten: Core, Traversal, Signature, Dynamics, Horizon, Cognition, Eval-Matrix, PhaseMatrix, Dual-Fabric Stitch und TPT-MTL. Die offene Grenze ist nicht weitere Theorie, sondern reproduzierbare Validierung. Der Nutzer soll nicht manuell Dutzende Befehle koordinieren, sondern einen einzigen Validierungsbefehl ausführen können.

1.3 Ziel

Der Validierungsrunner erzeugt aus einem Repository-Commit ein geschlossenes Validierungsartefakt:

$$\text{RepositoryCommit} + \text{ValidationProfile} + \text{OptionalDataset} \longrightarrow \text{ValidationSingularityBundle}$$

Dieses Bundle enthält:

- Repository-Snapshot und Environment-Fingerprint;
- Build-, Format-, Lint-, Test- und Doc-Ergebnisse;
- interne Benchmarks und Ground-Truth-Benchmarks;
- Eval-Matrix-Presets mit Plan, Run, Replay, Score und Report;
- optionale reale Offline-Domänenvalidierung;
- Layer-Ablations- und Marginal-Utility-Analyse;
- finale Klassifikation: Invalid, Diagnostic, PartialImprovement, EmpiricalImprovement;
- maschinenlesbare JSON-Reports und menschenlesbaren Markdown-Report.

One-Button-Invariante

Ein einzelner Befehl `pse-validate run -profile full` **MUST** genügen, um eine vollständige lokale Validierungsrunde auszuführen oder kontrolliert mit einem maschinenlesbaren Fehlerbericht zu stoppen.

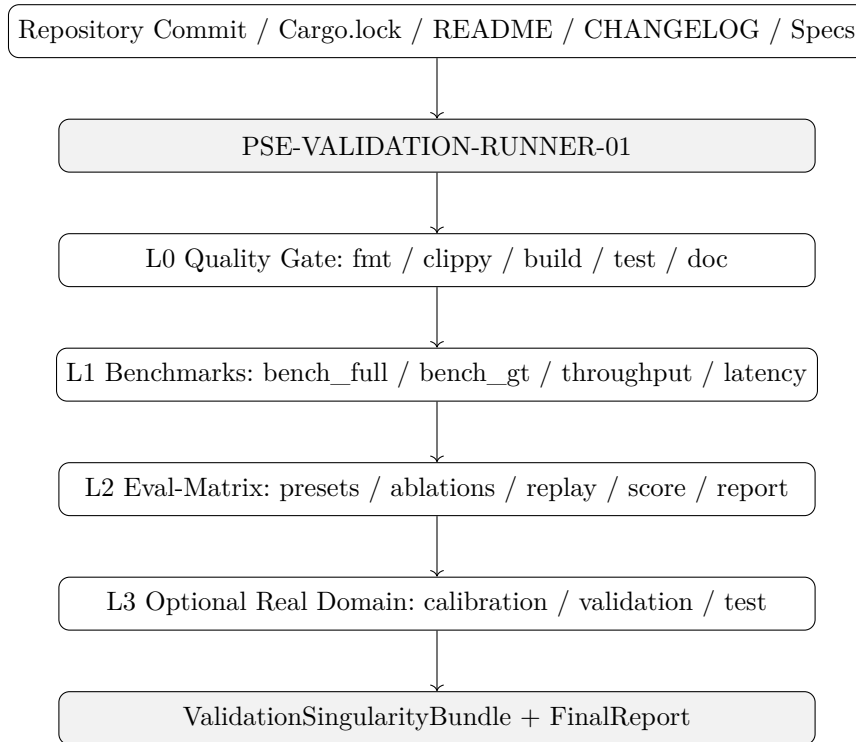
2 Nicht-Ziele

Der Runner ist nicht:

- eine neue PSE-Kognitionsschicht;
- ein Ersatz fuer die Eval-Matrix;
- ein neuer DomainAdapter;
- ein automatischer Beweis wirtschaftlicher Produktreife;
- ein Mechanismus, der schlechte Ergebnisse als Erfolg interpretiert;
- ein System, das Testdaten zur Kalibrierung verwenden darf.

Der Runner orchestriert, misst, bündelt und klassifiziert. Er erzeugt keine **SemanticCrystals**, keine **FinalizedEmissions** und keine neuen fachlichen Claims ohne Reportgrundlage.

3 Systemposition



3.1 Prinzip

Der Runner nutzt vorhandene CLI-Tools und Bibliotheken. Er darf bei Bedarf kleine Adapter fuer Pfad-, Log- und Report-Normalisierung implementieren. Er **MUST NOT** vorhandene Eval-Matrix-Logik duplizieren.

4 Konformitätsklassen

Klasse	Bedeutung
VR0	Repository-Snapshot und Quality Gate laufen deterministisch.
VR1	VR0 plus Core-Benchmarks und deterministische Replay-Prüfung.
VR2	VR1 plus interne Eval-Matrix-Presets und Layer-Ablation.
VR3	VR2 plus TPT/PhaseMatrix/Dual-Fabric-spezifische Presets, falls im Repository vorhanden.
VR4	VR3 plus reale Offline-Datendomäne mit Calibration/Validation/Test-Split.
VR5	VR4 plus vergleichende Produktvalidierung gegen Baselines und reproduzierbares Final Dossier.

Die erste Implementierung **MUST** mindestens VR3 erreichen. VR4/VR5 **MAY** vorbereitet, aber nicht erzwungen werden, wenn kein realer Datensatz vorhanden ist.

5 Ein-Knopf-Bedienmodell

5.1 Primärer Befehl

```
cargo run --release -p pse-validation-runner-cli -- run \
  --profile full \
  --out validation_runs/latest
```

Alternativer installierter Alias:

```
pse-validate run --profile full --out validation_runs/latest
```

5.2 Profile

Profil	Zweck
smoke	Minimaler schneller Durchlauf: repo snapshot, build, unit tests, ein Eval-Preset.
structural	Vollständige interne Strukturvalidierung ohne reale Datensätze.
full	Structural plus alle verfügbaren internen Benchmarks und Eval-Presets.
domain	Full plus ausgewählte reale Offline-Domäne mit Kalibrierung/Validierung/Test.
ci	CI-freundliche Variante mit Timeouts und maschinenlesbarem Exit-Code.

5.3 Subcommands

```
pse-validate init --out validation_profiles/default.json
pse-validate inspect --profile validation_profiles/default.json
pse-validate run --profile full --out validation_runs/latest
pse-validate resume --run validation_runs/<run_id>
pse-validate replay --run validation_runs/<run_id>
pse-validate score --run validation_runs/<run_id>
pse-validate report --run validation_runs/<run_id> --format md
pse-validate bundle --run validation_runs/<run_id> --out validation_bundle.zip
pse-validate verify-bundle validation_bundle.zip
```

6 Artefaktstruktur

Jeder Lauf **MUST** in einem eigenen Ordner liegen:

```
validation_runs/
  <run_id>_<commit_short>/
    manifest.json
    repo_snapshot.json
    environment.json
    command_plan.json
    command_log.jsonl
    quality/
      fmt.log
      clippy.log
      build.log
```

```

test.log
doc.log
quality_summary.json
bench/
  bench_full.log
  bench_gt.log
  throughput_summary.json
eval/
  agent-cognition/
    spec.json
    plan.json
    bundle.json
    replay.json
    summary.json
    summary.md
  streaming-event-detection/
  post-symbolic-ablation/
  phase-matrix-substrate/
  dual-fabric-stitch/
  topology-tpt-mtl/
domain/
  dataset_manifest.json
  calibration_report.json
  validation_report.json
  test_report.json
final/
  final_report.json
  final_report.md
  conclusion.json
  open_obligations.md

```

7 Datenmodelle

7.1 ValidationProfile

```

pub struct ValidationProfile {
  pub profile_id: Hash256,
  pub profile_name: String,
  pub conformance_target: ValidationConformanceClass,
  pub quality: QualityGateConfig,
  pub benchmarks: BenchmarkConfig,
  pub eval_matrix: EvalMatrixRunConfig,
  pub domain: Option<DomainValidationConfig>,
  pub timeouts: TimeoutConfig,
  pub output_policy: OutputPolicy,
}

pub enum ValidationConformanceClass { VR0, VR1, VR2, VR3, VR4, VR5 }

```

7.2 RepositorySnapshot

```

pub struct RepositorySnapshot {
  pub repo_root_hash: Hash256,
  pub git_commit: String,

```

```
pub git_branch: String,  
pub dirty: bool,  
pub cargo_lock_hash: Hash256,  
pub readme_hash: Option<Hash256>,  
pub changelog_hash: Option<Hash256>,  
pub workspace_manifest_hash: Hash256,  
}
```

7.3 EnvironmentFingerprint

```
pub struct EnvironmentFingerprint {  
    pub os: String,  
    pub arch: String,  
    pub rustc_version: String,  
    pub cargo_version: String,  
    pub cpu_model: Option<String>,  
    pub logical_cores: Option<u32>,  
    pub memory_bytes: Option<u64>,  
    pub timestamp_wallclock_excluded_from_hash: bool,  
}
```

Wall-clock-Zeit **MAY** im menschenlesbaren Report erscheinen, **MUST NOT** aber Teil von content-addressed Audit-Hashes sein.

7.4 ValidationCommand

```
pub struct ValidationCommand {  
    pub command_id: Hash256,  
    pub phase: ValidationPhase,  
    pub argv: Vec<String>,  
    pub working_dir: String,  
    pub env: BTreeMap<String, String>,  
    pub timeout_seconds: u64,  
    pub expected_exit: ExpectedExit,  
    pub output_paths: Vec<String>,  
}  
  
pub enum ValidationPhase {  
    Snapshot,  
    Quality,  
    Benchmark,  
    EvalMatrix,  
    DomainCalibration,  
    DomainValidation,  
    DomainTest,  
    Scoring,  
    Reporting,  
    Bundling,  
}
```

7.5 ValidationRunLedger

```
pub struct ValidationRunLedger {  
    pub ledger_id: Hash256,
```



```

    pub run_id: StableId,
    pub repo_snapshot_hash: Hash256,
    pub profile_hash: Hash256,
    pub entries: Vec<ValidationRunEntry>,
    pub chain_hash: Hash256,
}

pub struct ValidationRunEntry {
    pub entry_id: Hash256,
    pub sequence: u64,
    pub phase: ValidationPhase,
    pub command_hash: Hash256,
    pub exit_code: i32,
    pub stdout_hash: Hash256,
    pub stderr_hash: Hash256,
    pub artifact_hashes: Vec<Hash256>,
    pub previous_entry_hash: Option<Hash256>,
}

```

8 Ausführungsphasen

8.1 Phase L0: Snapshot und Quality Gate

Befehl	Zweck
git rev-parse HEAD	Commit-ID.
git status -porcelain	Dirty-State.
cargo fmt -all - -check	Formatprüfung.
cargo clippy -workspace	Lints.
-all-targets -locked	
RUSTFLAGS="-D warnings"	Build ohne Warnungen.
cargo build -workspace	
-all-targets -locked	
cargo test -workspace	Test-Suite.
-locked	
RUSTDOCFLAGS="-D warnings" cargo doc	Dokumentation ohne Warnungen.
-workspace -no-deps	
-locked	

Wenn L0 fehlschlägt, **MUST** der Runner die späteren Phasen standardmäßig überspringen und ein Urteil Invalid erzeugen. Eine Option `-continue-after-quality-fail` **MAY** Diagnostik erzwingen, **MUST** aber im Report klar markiert werden.

8.2 Phase L1: Core-Benchmarks

```

cargo run --release --example bench_full -p pse-core
cargo run --release -p pse-demo
cargo run --release -p pse-bench-gt --bin bench_gt

```

Der Runner **MUST** Durchsatz, Latenz, deterministische Replay-Indikatoren, Baseline-Vergleiche und Exit-Codes extrahieren, soweit sie maschinenlesbar oder per Log-Parser verfügbar sind.

8.3 Phase L2: Eval-Matrix-Presets

Für jedes Preset gilt die Sequenz:

```
pse-eval-matrix init --template <preset> --out spec.json
pse-eval-matrix validate --spec spec.json
pse-eval-matrix plan --spec spec.json --out plan.json
pse-eval-matrix run --spec spec.json --plan plan.json --out bundle.json
pse-eval-matrix replay --bundle bundle.json --out replay.json
pse-eval-matrix score --spec spec.json --bundle bundle.json --out summary.json
pse-eval-matrix report --summary summary.json --format md --out summary.md
```

Pflicht-Presets:

- agent-cognition;
- streaming-event-detection;
- post-symbolic-ablation;
- phase-matrix-substrate;
- dual-fabric-stitch.

Wenn TPT-MTL-Metriken vorhanden sind, aber kein Preset existiert, **MUST** der Runner einen klaren MissingPreset Befund erzeugen. Optional **MAY** eine Implementierung ein neues Preset topology-tpt-mtl ergänzen.

8.4 Phase L3: Optionale reale Domänenvalidierung

Ein Domain-Lauf ist nur gültig, wenn ein DatasetManifest mit drei Splits vorhanden ist:

$$D = D_{calibration} \cup D_{validation} \cup D_{test}, \quad D_{calibration} \cap D_{test} = \emptyset$$

Regel:

- Kalibrierung **MUST** ausschließlich auf Calibration-Split stattfinden.
- Auswahl von Schwellen/Profilen **MAY** auf Validation-Split stattfinden.
- Test-Split **MUST** frozen sein und darf nur einmal final ausgewertet werden.

9 Validierungsurteile

Urteil	Bedeutung
Invalid	Build, Replay, Ledger, Hashing oder Datensplit verletzt. Kein Befund verwertbar.
DiagnosticFinding	System läuft und erzeugt Reports, aber keine belastbare Verbesserung.
PartialImprovement	Verbesserung in Zielmetriken, aber Tradeoff bei Sicherheit, Replay oder Drift.
EmpiricalImprovement	Verbesserung nach definierter Schlussformel erfüllt.
ExternalValidationPending	Interne Validierung bestanden, aber reale Offline-Domäne fehlt.

9.1 Schlussformel

Eine empirische Verbesserung darf nur gemeldet werden, wenn:

$$\Delta U_{task} > 0 \wedge \Delta U_{safety} \geq 0 \wedge ReplayIdentity = 1 \wedge InvalidRunRate \leq \epsilon \wedge LMU_{target} > 0.$$

Wenn eine Bedingung fehlt oder nicht messbar ist, **MUST** das Urteil `DiagnosticFinding` oder `ExternalValidationPending` lauten.

10 Metrikgruppen

Gruppe	Beispiele
Quality	BuildPass, TestPassRate, ClippyPass, DocPass, DirtyRepo.
Performance	Throughput, MacroStepLatency, BenchFullDuration.
Replay	ReplayIdentity, LedgerChainValidity, HashStability.
Core Outcome	CrystalRate, FalseCrystalRate, GatePassRate, FalsifierPassRate.
Cognition	HandoffPassRate, PanoramaCoverage, ConstraintPercolation, MemoryReuse.
PhaseMatrix	ClusterFormationRate, DissolutionTracePreservation, BoundaryViolationRate.
DualFabric	StitchReplayIdentity, FabricHIsolationRate, TensorRevisionMonotonicity.
Topology	TopologyRobustness, MicroLiftCoverage, CarrierContinuity, TraceCompleteness.
Domain	Precision, Recall, F1, DetectionDelay, CalibrationLeakage, BaselineDelta.

11 Runner-Architektur

11.1 Crates und Tools

```
crates/pse-validation-runner/
src/
  lib.rs
  profile.rs
  snapshot.rs
  environment.rs
  command_plan.rs
  executor.rs
  ledger.rs
  parsers.rs
  eval_matrix.rs
  domain.rs
  scoring.rs
  report.rs
  bundle.rs
  replay.rs

tools/pse-validation-runner-cli/
src/main.rs
```

11.2 Feature Flags

Feature	Bedeutung
validation-runner	Kernlogik.
validation-cli	CLI.
validation-domain	Offline-Datensatzvalidierung.
validation-reports	Markdown/JSON-Reports.
validation-archive	ZIP/TAR-Bundles.
validation-github	Optional: CI-/GitHub-Metadatenimport, kein Pflichtteil.

12 Report-Schichten

12.1 FinalReport JSON

```
pub struct FinalValidationReport {
  pub report_id: Hash256,
  pub run_id: StableId,
  pub repo_snapshot_hash: Hash256,
  pub environment_hash: Hash256,
  pub profile_hash: Hash256,
  pub quality_summary: QualitySummary,
  pub benchmark_summary: BenchmarkSummary,
  pub eval_summary_refs: Vec<Hash256>,
  pub domain_summary: Option<DomainValidationSummary>,
  pub conclusion: ValidationConclusion,
  pub open_obligations: Vec<OpenObligation>,
}

pub enum ValidationConclusion {
  Invalid,
  DiagnosticFinding,
  PartialImprovement,
  EmpiricalImprovement,
  ExternalValidationPending,
}
```

12.2 FinalReport Markdown

Der Markdown-Report **MUST** fuer Menschen lesbar sein und folgende Gliederung besitzen:

1. Executive Summary;
2. Repository Snapshot;
3. Quality Gate;
4. Benchmarks;
5. Eval-Matrix Results;
6. Layer Marginal Utility;
7. Replay and Determinism;
8. Domain Validation, falls vorhanden;
9. Conclusion;
10. Open Obligations und Next Actions.

13 Fehlerklassen

```
pub enum ValidationFailureKind {  
    CommandFailed,  
    Timeout,  
    MissingBinary,  
    MissingPreset,  
    ReplayMismatch,  
    LedgerChainBroken,  
    DirtyRepository,  
    CalibrationLeakage,  
    TestSplitMissing,  
    MetricMissing,  
    ReportParseFailed,  
    NonDeterministicOutput,  
    BundleVerificationFailed,  
}
```

Jede Fehlerklasse **MUST** eine Recovery-Anweisung im Report erzeugen.

14 Erste reale Domänenprofile

14.1 Empfohlene Reihenfolge

1. **syslog-security**: klarer Eventcharakter, gute Offline-Logs, niedrigere Datenschutzlast.
2. **seismo**: natürliche Ereignisse, klare Zeitachsen, Baselines einfach.
3. **binance-regime**: gute Datenverfügbarkeit, aber mehr Rauschen.
4. **vitals-ecg**: hoher Nutzen, aber medizinische Sorgfalt und Datenlage komplexer.
5. **code-agent-repair**: wirtschaftlich relevant, aber Bewertungssetup schwieriger.

14.2 Minimaler DatasetManifest

```
pub struct DatasetManifest {  
    pub dataset_id: Hash256,  
    pub domain: String,  
    pub calibration_split: DatasetSplit,  
    pub validation_split: DatasetSplit,  
    pub test_split: DatasetSplit,  
    pub ground_truth_profile: GroundTruthProfile,  
    pub license: Option<String>,  
    pub provenance_hash: Hash256,  
}
```

15 Tests

15.1 Unit Tests

1. `profile_serializes_canonically`
2. `repo_snapshot_detects_dirty_state`
3. `ledger_chain_detects_modified_entry`

4. `command_plan_is_deterministic`
5. `missing_preset_yields_missing_preset_failure`
6. `final_conclusion_requires_replay_identity`
7. `external_validation_pending_without_domain_data`
8. `calibration_leakage_invalidates_domain_run`

15.2 Integration Tests

1. `smoke_profile_produces_bundle`
2. `structural_profile_runs_eval_presets`
3. `run_resume_continues_after_interruption`
4. `bundle_verify_detects_tampering`
5. `markdown_report_contains_required_sections`
6. `json_report_is_content_addressed`

15.3 CLI Smoke Tests

1. `cli_init_writes_profile`
2. `cli_inspect_reads_profile`
3. `cli_run_smoke_completes`
4. `cli_replay_validates_bundle`
5. `cli_report_writes_markdown`

16 Definition of Done

Eine Implementierung gilt als abgeschlossen, wenn:

1. `pse-validation-runner` und `pse-validation-runner-cli` im Workspace registriert sind;
2. `pse-validate run -profile smoke` lokal funktioniert;
3. `pse-validate run -profile structural` alle internen Eval-Presets ausführt oder fehlende Presets korrekt meldet;
4. jeder Lauf einen `validation_runs/<run_id>` Ordner erzeugt;
5. `final_report.json` und `final_report.md` erzeugt werden;
6. Replay und Bundle-Verifikation funktionieren;
7. keine Wall-clock-Zeit in content-addressed Hashes eingeht;
8. alle Command-Outputs ghasht und im Ledger referenziert werden;
9. `cargo fmt`, `clippy`, `build`, `test`, `doc` fuer den Runner grün sind;
10. README und CHANGELOG um den Runner ergänzt sind.

17 Coding-Agent-Auftrag

PATCH-ID: PSE-VALIDATION-RUNNER-01
TITLE: One-Button Validation Singularity

GOAL:

Implementiere einen Validierungsrunner fuer das aktuelle PSE-Repository.
Der Runner soll mit einem Befehl Build, Tests, Benchmarks, Eval-Matrix-Presets,

Replay, Scoring, Reports und Artefakt-Bundling ausfuehren.

PRIMARY COMMAND:

```
pse-validate run --profile full --out validation_runs/latest
```

MUST:

- neues crate crates/pse-validation-runner anlegen;
- neues CLI tool tools/pse-validation-runner-cli anlegen;
- Profile smoke, structural, full, domain, ci unterstuetzen;
- RepositorySnapshot und EnvironmentFingerprint erfassen;
- cargo fmt/clippy/build/test/doc orchestrieren;
- bench_full, pse-demo und bench_gt ausfuehren, soweit verfuegbar;
- Eval-Matrix-Presets agent-cognition, streaming-event-detection, post-symbolic-ablation, phase-matrix-substrate, dual-fabric-stitch ausfuehren;
- fehlende Presets als MissingPreset reporten, nicht verschweigen;
- optional topology-tpt-mtl Preset erkennen oder als offene Obligation melden;
- validation_runs/<run_id>/ mit manifest, ledger, logs, reports erzeugen;
- final_report.json und final_report.md erzeugen;
- Bundle-Erstellung und Bundle-Verifikation implementieren;
- deterministic hashing, sorted lists, BTreeMap und JCS-compatible serialization verwenden.

MUST NOT:

- keine neue Kognitionsschicht implementieren;
- keine Eval-Matrix-Logik duplizieren;
- keine nicht gemessene Erfolgsbehauptung erzeugen;
- keine Testdaten zur Kalibrierung verwenden;
- keine Wall-clock-Zeit in Audit-Hashes aufnehmen.

Definition of Done:

- cargo fmt clean;
- cargo clippy clean;
- cargo test --workspace green;
- docs build warning-free;
- pse-validate run --profile smoke erzeugt gueltiges Bundle;
- pse-validate verify-bundle erkennt manipulierte Reports.

18 Schlussformel

Der Validierungsrunner ist erfolgreich, wenn aus einem Commit nicht mehr nur ein Build entsteht, sondern ein reproduzierbarer Befund:

$$\text{Commit} \rightarrow \text{Build} \rightarrow \text{Test} \rightarrow \text{Benchmark} \rightarrow \text{Eval} \rightarrow \text{Replay} \rightarrow \text{Score} \rightarrow \text{Report} \rightarrow \text{Urteil}$$

Endinvariante

Kein Fortschritt gilt als validiert, solange er nicht als reproduzierbares Artefakt mit Commit, Environment, Commands, Logs, Reports, Replay, Score und Schlussurteil vorliegt.
